

CMU 10-703: Deep Reinforcement Learning

Lecture 2: Introduction to Reinforcement Learning

Max Simchowitz

Carnegie Mellon University

Reinforcement Learning from Consequences

Definition. Reinforcement learning is a framework for learning to make sequential decisions through interaction with an environment.

Supervised learning

Correct answers are provided.

Unsupervised learning

Structure must be found in data.

Reinforcement learning

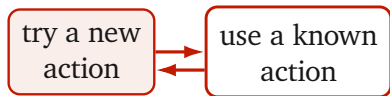
Rewards report consequences, not correct actions.

The agent must discover which choices lead to high return.

Source: Achiam et al., "Constrained Policy Optimization," ICML 2017.

Two Reinforcement Learning Challenges

Exploration versus exploitation



An agent must learn what actions do while still collecting reward.

Delayed consequences



Temporal credit assignment asks which earlier decisions produced a later reward.

1.

MDPs, Policies, Trajectories

CMU 10-703: Deep Reinforcement Learning

Markov Decision Processes

Markov decision process. An MDP is defined by the tuple

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma).$$

$\mathcal{S}$ state space

$r(s_t, a_t)$ scalar reward

$\mathcal{A}$ action space

$\gamma \in [0, 1]$ discount factor

$P(s_{t+1} \mid s_t, a_t)$ transition dynamics

$P(s_0)$ initial state distribution

For random $R(s, a)$, use its mean $r(s, a) := \mathbb{E}[R(s, a)]$.

The Markov Property

Markov property. Given the present state and action, the past adds no information about the next state.

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots) = P(s_{t+1} \mid s_t, a_t)$$

Practical test

Can two histories with the same state predict different futures?

If they can

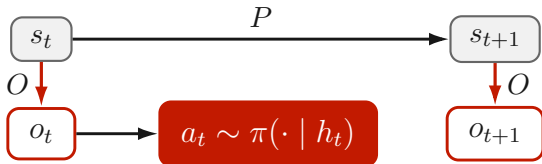
Add missing variables or recent observations to the state representation.

State representation is part of the model.

Partially Observable Markov Decision Processes

Partially observable MDP. The environment has a state s_t , but the agent receives an observation o_t .

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{O}, P, r, O, \gamma), \quad O(o_t \mid s_t, a_t)$$



The policy may need observation history rather than only o_t .

Policies May Use the Entire History

Policy. A policy maps the information available at time t to a distribution in $\Delta(\mathcal{A})$.

MDP

$$\pi_t : (s_{1:t}, a_{1:t-1}) \rightarrow \Delta(\mathcal{A})$$

The agent observes the state history.

POMDP

$$\pi_t : (o_{1:t}, a_{1:t-1}) \rightarrow \Delta(\mathcal{A})$$

The agent observes only the observation history.

A policy need not be a function of the current state or observation alone.

Markov Policies Use Only the Present

Markov policy. In an MDP, the action distribution depends only on the current state: $\pi(a_t \mid s_t)$.

MDP

The state contains all information needed to predict the future. Markov policies can attain the optimal return.

POMDP

A policy using only o_t can forget information that appeared in earlier observations.

Unless stated otherwise, “policy” will mean a Markov policy on an MDP.

History Can Be Promoted to State

Why memory matters

If you leave your keys in room A and move to room B, the current observation may no longer reveal where the keys are.

Context state

$$\bar{s}_t = (o_{1:t}, a_{1:t-1}).$$

Since $\bar{s}_t$ contains the complete previous context, its evolution is Markov.



Deterministic and Stochastic Markov Policies

Deterministic

$$\pi : \mathcal{S} \rightarrow \mathcal{A}, \quad a = \pi(s)$$

One action is selected with probability one.

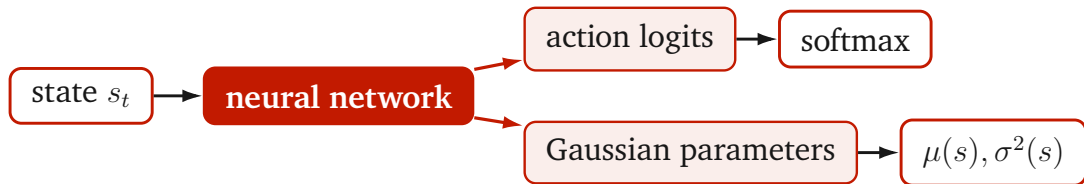
Stochastic

$$\pi(a \mid s), \quad \sum_{a \in \mathcal{A}} \pi(a \mid s) = 1$$

The policy specifies a distribution over actions.

Deterministic policies are the special case in which all mass is placed on one action.

Neural Network Policies



Discrete actions

Softmax converts network logits into action probabilities.

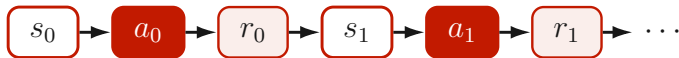
Continuous actions

A network can output the mean and variance of a Gaussian policy.

Trajectories and Episodes

Trajectory. A sequence of states, actions, and optionally rewards generated by interaction.

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, s_2, a_2, r_2, \dots)$$



Trajectory

The sequence itself, also called a rollout.

Episode

The session or container in which a trajectory occurs.

Trajectory Factorization

$$P_{\pi}(\tau) = P_{\pi}(s_0, a_0, s_1, a_1, \dots)$$

Start with the joint probability of a rollout.

Trajectory Factorization

$$\begin{aligned} P_{\pi}(\tau) &= P_{\pi}(s_0, a_0, s_1, a_1, \dots) \\ &= P(s_0) \prod_{t=0}^T P(s_{t+1} \mid s_t, a_t) \pi(a_t \mid s_t) \end{aligned}$$

Apply the Markov property and the policy conditional distribution.

Trajectory Factorization

$$\begin{aligned} P_{\pi}(\tau) &= P_{\pi}(s_0, a_0, s_1, a_1, \dots) \\ &= P(s_0) \prod_{t=0}^T P(s_{t+1} \mid s_t, a_t) \pi(a_t \mid s_t) \end{aligned}$$

Environment

$$P(s_0) \prod_{t=0}^T P(s_{t+1} \mid s_t, a_t)$$

Policy

$$\prod_{t=0}^T \pi(a_t \mid s_t)$$

We write $\mathbb{E}_{\tau \sim \pi}$ and $\mathbb{P}_{\tau \sim \pi}$ for expectation and probability under this trajectory law.

2.

Objectives and Returns

CMU 10-703: Deep Reinforcement Learning

Finite-Horizon Returns and Objectives

Return. For an episode with fixed horizon T ,

$$R(\tau) = \sum_{t=0}^T r(s_t, a_t).$$

Objective. Choose the policy with the largest expected return.

$$J_T(\pi) := \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T r(s_t, a_t) \right], \quad \pi^* \in \arg \max_{\pi} J_T(\pi).$$

This form fits tasks with a natural endpoint and weights every reward equally.

Discounted Infinite-Horizon Returns

Discounted return. For a continuing task,

$$R(\tau) = \sum_{t=0}^{\infty} \gamma^t r(s_t, a_t), \quad \gamma \in [0, 1].$$

$$J_{\gamma}(\pi) := \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right], \quad \pi^* \in \arg \max_{\pi} J_{\gamma}(\pi).$$

Convergence

Bounded infinite-horizon returns converge when $\gamma < 1$.

Time preference

Immediate rewards receive more weight.

Future uncertainty

Later events can receive less weight.

Why Geometric Discounting?

Exponential decay is equivalent

$$e^{-\alpha t} = \gamma^t \quad \text{when} \quad \gamma = e^{-\alpha}.$$

Geometric discounting is the discrete-time form of exponential decay.

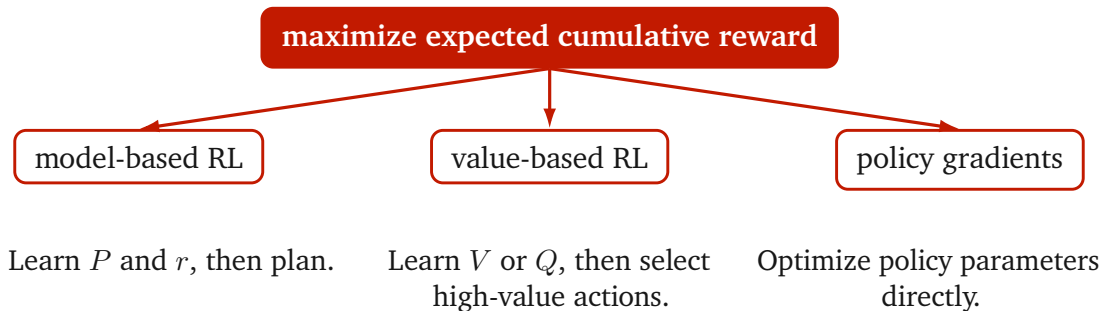
One step factors cleanly

$$\gamma^{t+1} = \gamma \cdot \gamma^t.$$

After one transition, the remaining return has the same form, scaled by γ .

This recursive factorization is the algebraic step that enables the Bellman equations.

Three Policy Extraction Methods



The objective stays fixed; the route to optimizing it changes.

3.

Value Functions

CMU 10-703: Deep Reinforcement Learning

Value Functions

State value. The expected discounted return from state s when the agent follows policy π .

$$V^{\pi}(s) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s \right].$$

Fixed

The policy π

Varies

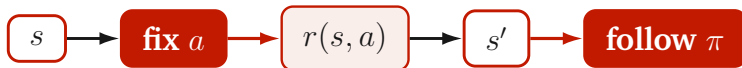
The starting state s

Higher value means more expected future reward under the fixed policy.

Action-Value Functions

Action value. The expected return from state s after taking action a , then following policy π .

$$Q^{\pi}(s, a) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a \right].$$



Q^{π} compares actions available at the same state.

Deriving Q^π from V^π

$$Q^\pi(s, a) = r(s, a) + \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=1}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a \right]$$

Separate the immediate reward from the remaining return.

Deriving Q^π from V^π

$$\begin{aligned} Q^\pi(s, a) &= r(s, a) + \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=1}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a \right] \\ &= r(s, a) + \mathbb{E}_{s' \sim P(\cdot | s, a)} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=1}^{\infty} \gamma^t r(s_t, a_t) \mid s_1 = s' \right] \end{aligned}$$

Condition on $s_1 = s'$; the Markov property removes the earlier history.

Deriving Q^π from V^π

$$\begin{aligned} Q^\pi(s, a) &= r(s, a) + \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=1}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a \right] \\ &= r(s, a) + \mathbb{E}_{s' \sim P(\cdot | s, a)} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=1}^{\infty} \gamma^t r(s_t, a_t) \mid s_1 = s' \right] \\ &= r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} [V^\pi(s')]. \end{aligned}$$

Reindex time and factor $\gamma^{t+1} = \gamma \gamma^t$.

Deriving V^π from Q^π

$$V^\pi(s) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s \right]$$

Start from the state-value definition.

Deriving V^π from Q^π

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s \right] \\ &= \mathbb{E}_{a \sim \pi(\cdot|s)} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a \right] \end{aligned}$$

Apply the tower rule by first conditioning on the initial action.

Deriving V^π from Q^π

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s \right] \\ &= \mathbb{E}_{a \sim \pi(\cdot|s)} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \mid s_0 = s, a_0 = a \right] \\ &= \mathbb{E}_{a \sim \pi(\cdot|s)} [Q^\pi(s, a)] . \end{aligned}$$

State value is the policy average of action value.

V^π and Q^π Average Different Uncertainty

Average over policy actions

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot|s)}[Q^\pi(s, a)].$$

Average over future states

$$Q^\pi(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)}[V^\pi(s')].$$

Substitute either identity into the other to express V^π or Q^π recursively in terms of itself.

4.

Bellman Equations

CMU 10-703: Deep Reinforcement Learning

Bellman Equation for State Values

$$V^{\pi}(s) = \mathbb{E}_{a \sim \pi(\cdot|s)} [Q^{\pi}(s, a)]$$

Average over the action selected by the policy.

Bellman Equation for State Values

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{a \sim \pi(\cdot|s)} [Q^\pi(s, a)] \\ &= \mathbb{E}_{a \sim \pi(\cdot|s)} [r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s, a)} [V^\pi(s')]] \end{aligned}$$

Expand the action value into immediate reward and successor value.

Bellman Equation for State Values

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_{a \sim \pi(\cdot | s)} [Q^\pi(s, a)] \\ &= \mathbb{E}_{a \sim \pi(\cdot | s)} [r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} [V^\pi(s')]] \\ &= \sum_{a \in \mathcal{A}} \pi(a | s) \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) V^\pi(s') \right]. \end{aligned}$$

For finite spaces, write both expectations as sums.

Bellman Equation for Action Values

$$Q^{\pi}(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} [V^{\pi}(s')]$$

The first action is fixed; later actions follow π .

Bellman Equation for Action Values

$$\begin{aligned} Q^\pi(s, a) &= r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} [V^\pi(s')] \\ &= r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} \left[\sum_{a' \in \mathcal{A}} \pi(a' | s') Q^\pi(s', a') \right] \end{aligned}$$

Replace next-state value by a policy average of Q^π .

Bellman Equation for Action Values

$$\begin{aligned} Q^\pi(s, a) &= r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} [V^\pi(s')] \\ &= r(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot | s, a)} \left[\sum_{a' \in \mathcal{A}} \pi(a' | s') Q^\pi(s', a') \right] \\ &= r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' | s, a) \sum_{a' \in \mathcal{A}} \pi(a' | s') Q^\pi(s', a'). \end{aligned}$$

For finite spaces, expand the remaining expectations.

Bellman Optimality Equation for State Values

$$V^*(s) = \max_{a \in \mathcal{A}} Q^*(s, a)$$

The optimal policy selects an action with maximal Q^* .

Bellman Optimality Equation for State Values

$$\begin{aligned} V^*(s) &= \max_{a \in \mathcal{A}} Q^*(s, a) \\ &= \max_{a \in \mathcal{A}} \left[r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s, a) V^*(s') \right]. \end{aligned}$$

Substitute the best action into the one-step state-value equation.

Bellman Optimality Equation for Action Values

$$Q^*(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s, a) \max_{a' \in \mathcal{A}} Q^*(s', a').$$

At the next state, optimal continuation selects the best action.

Bellman Optimality Equation for Action Values

$$Q^*(s, a) = r(s, a) + \gamma \sum_{s' \in \mathcal{S}} P(s' \mid s, a) \max_{a' \in \mathcal{A}} Q^*(s', a').$$

Choose any $\hat{a}(s) \in \arg \max_{a' \in \mathcal{A}} Q^*(s, a')$

$$\pi^*(a \mid s) = \begin{cases} 1, & a = \hat{a}(s), \\ 0, & \text{otherwise.} \end{cases}$$

A greedy policy places all probability on an action with maximal Q^* .

Why an Optimal Policy Must Be Greedy

Heuristic assumptions

The initial distribution has full support, and a stationary optimal policy π^* exists and is optimal even against time-dependent policies.

$$J(\pi^*) = \mathbb{E}_{s_0 \sim P_0}[V^{\pi^*}(s_0)] = \mathbb{E}_{s_0 \sim P_0} \mathbb{E}_{a_0 \sim \pi^*(\cdot|s_0)}[Q^{\pi^*}(s_0, a_0)].$$

Why an Optimal Policy Must Be Greedy

Heuristic assumptions

The initial distribution has full support, and a stationary optimal policy π^* exists and is optimal even against time-dependent policies.

$$J(\pi^*) = \mathbb{E}_{s_0 \sim P_0}[V^{\pi^*}(s_0)] = \mathbb{E}_{s_0 \sim P_0} \mathbb{E}_{a_0 \sim \pi^*(\cdot | s_0)}[Q^{\pi^*}(s_0, a_0)].$$

If π^* assigned positive probability to a strictly suboptimal action at any state, changing that one choice would increase J . Hence $\pi^*(a | s) > 0 \Rightarrow a \in \arg \max_{a'} Q^{\pi^*}(s, a')$.

Bellman Recursion in RL Algorithms

Long-horizon evaluation becomes a sequence of one-step predictions.



Consistency

Values agree with their one-step predictions.

Dynamic programming

Value and policy iteration.

TD learning

Learn from one-step transitions.

Bootstrapping

Estimate values from other value estimates.

5.

Occupancy Measures

CMU 10-703: Deep Reinforcement Learning

State and State–Action Occupancy

Discounted occupancy. The discounted frequency with which a policy visits a state or state–action pair.

State occupancy

$$\rho^\pi(s) = \sum_{t=0}^{\infty} \gamma^t \mathbb{P}_{\tau \sim \pi}(s_t = s).$$

State–action occupancy

$$\rho^\pi(s, a) = \sum_{t=0}^{\infty} \gamma^t \mathbb{P}_{\tau \sim \pi}(s_t = s, a_t = a).$$

These formulas use probabilities for discrete spaces; continuous spaces use occupancy densities.

Occupancy Measures Determine the Policy

$$\rho^\pi(s, a) = \sum_{t=0}^{\infty} \gamma^t \mathbb{P}_{\tau \sim \pi}(s_t = s, a_t = a)$$

A policy uniquely determines its occupancy measure.

Occupancy Measures Determine the Policy

$$\begin{aligned}\rho^\pi(s, a) &= \sum_{t=0}^{\infty} \gamma^t \mathbb{P}_{\tau \sim \pi}(s_t = s, a_t = a) \\ &= \sum_{t=0}^{\infty} \gamma^t \mathbb{P}_{\tau \sim \pi}(s_t = s) \pi(a \mid s) = \rho^\pi(s) \pi(a \mid s)\end{aligned}$$

At a visited state, the action probability factors from state visitation.

Occupancy Measures Determine the Policy

$$\begin{aligned}\rho^\pi(s, a) &= \sum_{t=0}^{\infty} \gamma^t \mathbb{P}_{\tau \sim \pi}(s_t = s, a_t = a) \\ &= \sum_{t=0}^{\infty} \gamma^t \mathbb{P}_{\tau \sim \pi}(s_t = s) \pi(a \mid s) = \rho^\pi(s) \pi(a \mid s) \\ \pi(a \mid s) &= \frac{\rho^\pi(s, a)}{\rho^\pi(s)} \quad \text{whenever } \rho^\pi(s) > 0.\end{aligned}$$

If every state has positive occupancy, the occupancy measure also recovers the policy.

The Objective Is Linear in Occupancy

$$J_{\gamma}(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right]$$

Start from the trajectory objective.

The Objective Is Linear in Occupancy

$$\begin{aligned} J_\gamma(\pi) &= \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] \\ &= \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} r(s, a) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}_{\tau \sim \pi}(s_t = s, a_t = a) \end{aligned}$$

Regroup reward contributions by state–action pair.

The Objective Is Linear in Occupancy

$$\begin{aligned} J_\gamma(\pi) &= \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right] \\ &= \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} r(s, a) \sum_{t=0}^{\infty} \gamma^t \mathbb{P}_{\tau \sim \pi}(s_t = s, a_t = a) \\ &= \sum_{s \in \mathcal{S}} \sum_{a \in \mathcal{A}} \rho^\pi(s, a) r(s, a). \end{aligned}$$

Lifting from policies to feasible occupancy measures makes the reward objective linear.

Uses of Occupancy Measures



A normalized discounted occupancy distribution is $(1 - \gamma)\rho^\pi$ when $\gamma < 1$.

Exploration

Which states does the policy visit?

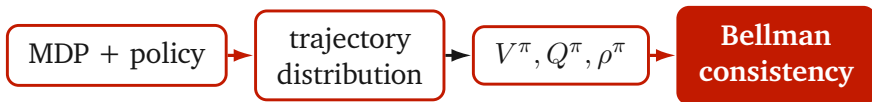
Policy gradients

Under which state distribution is the policy optimized?

Reward design

Which visited states receive reward?

Reinforcement Learning Framework



Behavior

$\pi(a | s)$ selects actions.

Objective

Expected return ranks policies.

Summaries

Values and occupancy compress trajectories.

Recursion

Bellman equations reduce the future to one step.

Topics in Subsequent Lectures

1. Dynamic programming
2. Policy gradients
3. Value-based learning
4. Actor–critic methods
5. Model-based reinforcement learning

Each method changes what is represented or optimized; all use the same MDP, return, value, and trajectory framework introduced here.